

RETRO SNAKE GAME

CSE444 — Software Engineering II

Take-Home Exam: Vibe Coding Game Development

This report documents the design, implementation, and reflection process for the **Retro Snake Game** — a Python/PyGame project developed as part of the CSE444 Take-Home Exam using *vibe coding* techniques. The game faithfully recreates the classic Snake arcade experience with modern polish: particle effects, animated food, colour-gradient snake body, progressive difficulty, and persistent high scores.

1. Introduction

1.1 Project Description

The Retro Snake Game is a faithful yet modernised recreation of the classic Snake arcade game, originally popularised in the 1970s and later on Nokia mobile phones. The player controls a continuously moving snake on a grid-based playing field, steering it to consume food pellets while avoiding collisions with the walls and the snake's own body. Each food item consumed increases the snake's length, progressively making navigation harder.

The game was developed in **Python 3.13** using the **PyGame 2.6** library. It targets desktop platforms (Windows, Linux, macOS) and requires no external assets — all graphics are rendered procedurally using PyGame primitives. Gameplay is rendered at 60 FPS with delta-time-based logic to ensure consistent behaviour regardless of hardware.

1.2 Objectives

- Implement a fully playable, self-contained Snake game in Python/PyGame.
- Apply *vibe coding*: iteratively discover and add features rather than planning everything upfront.
- Produce clean, modular, and well-commented code suitable for future maintenance.
- Provide an engaging retro aesthetic with particle effects, animated food, and a colour-gradient snake.
- Include a persistent high-score mechanism and progressive difficulty scaling.
-

2. Functional Requirements

The following table lists the 25 functional requirements identified during development. Requirements emerged iteratively — some were planned upfront, others surfaced during play-testing.

ID	Requirement
FR-01	The system shall display a grid-based game environment (40×29 cells) for the snake.
FR-02	The system shall allow the player to control the snake's direction using Arrow Keys or WASD.
FR-03	The system shall prevent the player from reversing the snake 180° in a single move.
FR-04	The system shall continuously move the snake in the current direction on a fixed tick interval.
FR-05	The system shall increase the snake's length by one segment upon consuming normal food.
FR-06	The system shall increase the snake's length by one segment upon consuming bonus food.
FR-07	The system shall generate normal food at random positions within the grid.
FR-08	The system shall ensure that normal food does not spawn on any segment of the snake.
FR-09	The system shall randomly spawn a bonus (gold) food item with a 25% chance after each normal food is eaten.
FR-10	The system shall remove bonus food from the screen after 8 seconds if not consumed.
FR-11	The system shall display a countdown timer above the bonus food item.
FR-12	The system shall blink the bonus food item when fewer than 3 seconds remain on its timer.
FR-13	The system shall detect collisions between the snake's head and the game boundaries (wall death).
FR-14	The system shall detect collisions between the snake's head and its own body (self-collision death).
FR-15	The system shall award 10 points to the player for each normal food item consumed.
FR-16	The system shall award 30 points to the player for each bonus food item consumed.
FR-17	The system shall display the current score in the HUD at the top of the screen.
FR-18	The system shall display the all-time high score in the HUD.
FR-19	The system shall increase the snake's movement speed as the score increases (every 10 pts).
FR-20	The system shall cap movement speed at a maximum of 22 moves per second.
FR-21	The system shall display the current difficulty level (derived from speed) in the HUD.
FR-22	The system shall spawn a particle burst effect at the position of consumed food.
FR-23	The system shall provide a pause/resume functionality accessible with the P key.
FR-24	The system shall provide a game-over screen showing final score, high score, and restart prompt.
FR-25	The system shall persist the high score to disk (JSON file) between sessions.

3. Approach to the Problem — Vibe Coding

3.1 What is Vibe Coding?

Vibe coding is an experimental development style that prioritises momentum, intuition, and rapid iteration over exhaustive upfront design. Rather than writing a full specification before touching code, the developer starts with a minimal working prototype and lets the design emerge organically through play and experimentation. This approach aligns naturally with game development, where the 'feel' of a game cannot be fully anticipated on paper — it must be played to be discovered.

3.2 Iterative Development Phases

Phase 1 — Skeleton (Day 1–2)

The very first version was a bare-bones loop: a white square moving rightward across a black screen using a list of (x, y) tuples. There was no food, no score, and no death detection. The goal was simply to see something moving. This gave immediate visual feedback and a foundation to build on.

Phase 2 — Core Mechanics (Day 3–4)

Added keyboard direction control, wall collision, self-collision, and a single food pellet that respawns on a random cell. The snake grew by one segment on each food eaten. At this stage the game was technically 'complete' but felt sterile and unpolished.

Phase 3 — Game States (Day 5)

A state machine (MENU → PLAYING → PAUSED → GAME_OVER) was introduced. This was a pivotal refactor: before it, restarting required killing and rerunning the script. Overlays for each state were added with keyboard prompts.

Phase 4 — Juice and Polish (Day 6–8)

This is where the 'vibe' part truly showed. Features added experimentally: colour gradient on the snake body (lerp from head colour to tail colour), a pulsing animation on food, particle burst effects on food consumption, a retro green-on-black colour palette. None of these were in any initial plan — they emerged from asking 'what would make this feel better?'

Phase 5 — Bonus Food & Difficulty (Day 9–10)

Bonus food with a countdown timer was added to give risk/reward moments. Speed scaling was tuned through repeated play-testing; the initial formula was too aggressive (game became unplayable at score 50). The final formula adds 0.7 moves/s per 10 points with a cap of 22 moves/s.

Phase 6 — Persistence & Cleanup (Day 11–12)

High score persistence via a JSON file was added. Code was refactored into clearly separated classes (Snake, Food, BonusFood, Particle, Game). Docstrings and inline comments were written. README was drafted.

3.3 Experiments and Changes

- Tried **pygame.sprite.Group** for entity management — abandoned in favour of simple lists and manual draw calls, which were easier to control for this scale of project.
- Experimented with **wrapping mode** (snake exits one side, enters the other) — removed because it made wall collisions irrelevant and reduced tension.

- Tested **sound effects** using `pygame.mixer.Sound` with generated sine-wave beeps — left out of the final build to keep the project dependency-free and focus on visual polish.
- Tried a **lives system** (3 lives before game over) — removed because it made the game feel too forgiving and reduced the tension of a single-life run.
- **Direction queueing**: initially only one direction was stored. Changed to a queue approach so fast key inputs are not dropped.

3.4 Challenges and Solutions

- **Self-collision false positives**: The snake's tail had not yet moved before collision checking, causing false self-collisions. Fixed by excluding the last body segment (tail tip) from the collision set, since it will vacate its cell on this tick.
- **Speed inconsistency across hardware**: Initial implementation moved the snake every N frames, causing different speeds on different machines. Fixed by using delta-time accumulation: `move_timer += dt`, trigger move when `timer >= 1/speed`.
- **Bonus food spawning on snake body**: BonusFood could initially spawn inside the snake. Fixed by filtering the full cell list to exclude all snake body segments before sampling.
- **Direction reversal bug**: Pressing the opposite key twice rapidly caused death. Fixed with the `OPPOSITE` dictionary that blocks any direction equal to the reverse of the current direction.

4. Implementation Overview

4.1 Architecture

The codebase is contained in a single file (**snake_game.py**, ~370 lines) and is organised into five classes plus a set of module-level utility functions and constants.

Component	Responsibility
Snake	Stores body deque, direction, movement logic, self/wall collision, rendering
Food	Random respawn on free cells, pulsing draw animation
BonusFood	Time-limited bonus pellet, countdown draw, blink when expiring
Particle	Single particle: position, velocity, fade; used for food-eaten burst
Game	State machine, event loop, HUD, overlay screens, score/high score, timing
Utilities	load/save_high_score, cell_rect, lerp_color (module-level functions)

4.2 Game Loop

The game loop runs at a target of 60 FPS. On each iteration: (1) **clock.tick(FPS)** returns the elapsed milliseconds, converted to seconds (dt); (2) events are polled; (3) game logic is updated via **Game.update(dt)**; (4) the screen is redrawn via **Game.draw()**. Snake movement is decoupled from the frame rate using a move timer. dt is accumulated and a grid move fires when the accumulator exceeds 1/speed seconds, guaranteeing consistent speed on all hardware.

4.3 Key Algorithms

- **Snake movement:** The body is a Python list, head at index 0. On each move: (1) compute new head position; (2) check wall/self collision; (3) insert new head at front; (4) pop the tail. On growth (food eaten), the tail is duplicated instead of popped via snake.grow().
- **Food spawning:** All COLS×ROWS cells are filtered against the snake body set. A random choice is made from the remaining free cells. This is O(N) but acceptable for the grid sizes used (≤1160 cells).
- **Speed scaling:** speed = BASE_SPEED + floor(score / 10) × SPEED_PER_10, capped at MAX_SPEED. With BASE=7, STEP=0.7, the game reaches 14 moves/s at score 100 and is hard-capped at 22 moves/s.
- **Colour gradient:** lerp_color(C_SNAKE_B, C_SNAKE_T, t) produces a smooth green-to-dark gradient along the body, where t = segment_index / (length - 1).

4.4 Tools and Technologies

Tool / Library	Version	Purpose
Python	3.13	Primary language
PyGame	2.6.1	Graphics, input, timing
reportlab	4.x	PDF report generation
json	stdlib	High score persistence
math	stdlib	sin/cos for particle and pulse animations

5. Discussion

5.1 What Worked Well

- Delta-time movement decoupling ensured the game ran consistently across debug and release builds at different FPS.
- The state machine approach kept each screen (menu, playing, paused, game over) cleanly separated and made adding new states trivial.
- Vibe coding was highly effective for the visual polish phase: trying things interactively (particle effects, colour gradients) produced better results faster than specifying them upfront would have.
- Using a plain Python list for the snake body (rather than a deque or sprite group) kept collision checks simple — 'in' on a list is perfectly fast for the small body sizes encountered in a single game session.
- The pre-rendered background surface (`_bg`) improved performance by avoiding redrawing the static grid on every frame.

5.2 What Did Not Work / Limitations

- **No sound:** time constraints meant audio was not implemented. A short 8-bit beep on food consumption would significantly improve the retro feel.
- **Single-file structure** works for this scale but would become unwieldy if the project were extended with multiple game modes or levels.
- **Food spawning algorithm** becomes slower as the snake grows very long, because it filters the entire cell list. A set-difference approach would be $O(1)$.
- The game window size is hard-coded; no fullscreen or resolution scaling is supported.

5.3 Lessons Learned

- Vibe coding works best when alternating between short bursts of building and stepping back to play. The feel of the game is as important as its correctness.
- Separating concerns early (Snake knows nothing about the HUD; Game knows nothing about particle physics) paid off later when changing one class didn't cascade into others.
- Delta-time architecture should be the default for any game, not an afterthought.
- Writing comments while coding — not after — is faster and produces more accurate documentation, because the intent is fresh.

5.4 Possible Improvements and Future Extensions

- **Sound effects** — 8-bit beeps using `pygame.mixer` with procedurally generated tones.
- **Multiple food types** — e.g. a 'speed boost' pellet that temporarily increases speed, or a 'shrink' pellet that reduces body length.
- **Obstacle tiles** — static walls randomly placed on the grid for added difficulty.
- **Two-player local co-op** — two snakes sharing the same board, controlled by different key sets.
- **Animated skin selector** — allow the player to choose between different snake colour themes from the menu.
- **Leaderboard screen** — display the top 5 session scores using in-memory storage.
- **Wrap-around mode toggle** — optional wall-wrap gameplay for a different challenge.
- **Grid size options** — 3×3 / 4×4 cell sizes to change game density.

References

PyGame documentation — <https://www.pygame.org/docs/>

Python 3.13 documentation — <https://docs.python.org/3/>

reportlab User Guide — <https://www.reportlab.com/docs/reportlab-userguide.pdf>

Classic Asteroids / Snake game mechanics — Wikipedia, Arcade Game Archives

Vibe Coding concept — iterative, intuition-led development methodology